Quarry: Deployment and Operations

Manoj Abhiram

November 10, 2026

CONTENTS

1. Runtime topology 3

2. Prerequisites 4

3. Configuration 4

4. First deployment 5

5. Local development 6

6. Schema migrations 6

7. Backup and restore 6

8. Scheduled work 8

9. Monitoring 9

10. Capacity and limits 10

11. Runbooks 10

- 11.1 The API will not start 10
- 11.2 Runs stay Pending 11
- 11.3 Leads show “Enrichment failed” 11
- 11.4 A mailbox stopped sending 11
- 11.5 Payments did not credit 11
- 11.6 Handle an opt-out or erasure request 11

12. Release procedure 12

13. Pre-hosting checklist 12

Document	How to build, configure, deploy, run, back up, upgrade and troubleshoot Quarry
Version	1.0
Date	2026-11-10
Audience	Whoever runs the service (today, the founder)

1. Runtime topology

Quarry ships as two containers defined in `docker-compose.yml`.

Container	Image source	Port	Role
`api`	`apps/api/Dockerfile` (multi-stage, Python 3.13 slim, non-root uid 10001)	8000	FastAPI, job worker, periodic scheduler
`web`	`apps/web/Dockerfile`	8080	nginx serving the static React build and proxying `/api` to `api`

State lives in one named volume, `quarry-data`, mounted at `/data`, holding `quarry.db` (plus the SQLite `-wal` and `-shm` files). Nothing else is persistent.

Hard rule: run exactly one API process. The job queue claim, the periodic scheduler, the rate limiter and the SQLite write lock are all in-process. Starting a second API instance against the same database causes duplicate sends, duplicate runs and write contention. The Dockerfile starts uvicorn with a single worker on purpose.

2. Prerequisites

Need	Why
Docker and docker compose	Production-like run
A Serper API key	Search (users may bring their own)
At least one LLM provider key (OpenRouter, Groq, or Anthropic)	Extraction and scoring
A domain and a TLS-terminating reverse proxy or load balancer	Cookies are `Secure` over https; nginx here listens on plain HTTP
An SMTP account for system email	Password reset and team invites
Optional: Stripe or Razorpay, Google OAuth client, HubSpot app, Hunter/Snov/PDL keys	Billing, Google sign-in, CRM, waterfall enrichment

3. Configuration

1. Copy `.env.production.example` to `.env.production`. It documents every variable the app reads. Real values go only in `.env.production`, which is git-ignored.
2. Set at minimum `SECRET_KEY` (generate with `openssl rand -hex 32`) and `SERPER_API_KEY`. The API refuses to start without them.
3. For a public deployment set:

Variable	Value
`FRONTEND_URL`	Public web URL (https)
`PUBLIC_API_URL`	Public API base URL; used in unsubscribe links, so it must be reachable by email recipients
`ALLOWED_ORIGINS`	The web origin(s)
`COOKIE_SECURE`	`true`
`COOKIE_SAMESITE`	`lax` when web and API share a site; `none` (forces Secure) when they are on different sites
`OUTREACH_POSTAL_ADDRESS`	Real sender postal address (required for compliant sending)
`PAYMENT_PROVIDER`	`stripe` or `razorpay`; leave `mock` only for local demos
`GOOGLE_REDIRECT_URI`	Registered with Google if Google sign-in is on

4. If web and API are on different sites, build the web image with `--build-arg VITE_API_URL=https://api.example.com/api`.

Never commit `.env.production`, API keys or the SQLite file. Keep a secret manager copy of `SECRET_KEY`: losing or rotating it makes stored credentials (mailbox passwords, provider keys) unreadable.

4. First deployment

```
cp .env.production.example .env.production      # fill it in
docker compose up --build -d
curl http://localhost:8000/api/health            # expect 200
open http://localhost:8080                       # web app, proxies /api
```

On first start the API creates the database, applies schema migrations, registers job handlers, starts the worker and scheduler, and marks orphaned `Pending` requests as failed (except those with a queued job).

Put TLS in front: point your proxy at port 8080 and forward `X-Forwarded-Proto`. Do not expose 8080 or 8000 directly to the internet.

5. Local development

```
# API (http://localhost:8000, Swagger at /docs)
cd apps/api && cp .env.example .env && uv sync
uv run uvicorn app.main:app --reload

# Web (http://localhost:3000, proxies /api)
cd apps/web && npm install && npm run dev
```

The dev web server also serves a component showcase at `/ui-lab`. Do not run `--reload` with a real queue of jobs you care about; a hot reload restarts the worker.

6. Schema migrations

- Migrations run automatically at API startup, before the app serves requests, in one transaction. A failure rolls back and the API does not start.
- Versions are recorded in `schema_migrations`. Version 1 is the baseline.
- To change the schema, append a migration with the next consecutive version in `app/db/migrations.py`; never edit or reorder a released one. Add a test beside `tests/db/test_migrations.py`.
- Startup logs a `Schema drift` warning if a model column is missing in the database, meaning a migration was forgotten.

Before upgrading a database that holds real data, back up the volume.

7. Backup and restore

SQLite in WAL mode is safe to copy only with the right method.

Method	Command	Notes
Online backup (recommended)	`docker compose exec api python -c "import sqlite3; s=sqlite3.connect('/data/quarry.db'); d=sqlite3.connect('/data/backup.db'); s.backup(d)"` then copy `backup.db` out	Consistent while the app runs
Cold copy	`docker compose stop api`, copy the volume contents including `-wal` and `-shm`, `docker compose start api`	Simple, short downtime

Restore. Stop `api`, replace `/data/quarry.db` (remove stale `-wal` and `-shm`), start `api`. Test a restore at least once before relying on backups.

Schedule. Daily backups with 14 days of retention is a sensible start. Keep backups encrypted and in a different location from the volume. Remember backups contain third-party personal data: erasure requests apply to them too, so keep retention short and document it.

8. Scheduled work

Work	Cadence	Mechanism
Send due outreach steps	Every minute	In-process scheduler enqueues `outreach.send_due`
Poll mailboxes for replies and bounces	Every 5 minutes	`outreach.poll_replies`
Deliver and retry webhooks	Every minute	`webhooks.deliver`
Signal scan, saved-search alerts	Every 6 hours	`signals.scan`, `searches.run`
Retention purge	Run daily	`python -m app.compliance.retention [--days 30] [--dry-run]` or `POST /api/compliance/retention/run`; schedule it with host cron
Expired session cleanup	At startup	`connect_db()`

Set `P3_SCHEDULER=false` to stop the periodic loop (for example during maintenance).

9. Monitoring

Signal	How to check
Liveness	`GET /api/health`; the container HEALTHCHECK calls it every 30 s
Job health	`GET /api/jobs?status=failed`; watch for growing `queued` and old `running` rows
Run outcomes	Request `status` and `failure_reason` in the app
Spend	`GET /api/usage/summary` (estimated, not billed)
Mailbox health	Mailbox `paused` and `pause_reason`; bounce rate in sequence stats
Webhook health	Delivery log per webhook
Logs	`docker compose logs -f api`

Gap: there is no metrics, tracing or paging system. Before hosting for others, add: request latency and error rate, queue depth and oldest queued job age, job failure rate, disk usage of the volume, and daily LLM and Serper spend. A free uptime monitor on `/api/health` is the minimum.

10. Capacity and limits

Limit	Value	Action when reached
API processes	1	Move to Postgres before adding more
Concurrent jobs	`JOB_WORKER_CONCURRENCY` (4)	Raise cautiously; writes still serialize
`num_leads` per request	200	Hard validation
Plan quotas	Free 3, Pro 50, Team 250 requests per month	Upgrade or top up credits
Per-user cost quota	$20 estimated per month	Override per user
Mailbox daily cap	40 default	Per mailbox setting
Disk	The volume	Monitor; SQLite grows with leads and the search cache

11. Runbooks

11.1 The API will not start

Symptom in logs	Cause	Fix
Missing `SECRET_KEY` or `SERPER_API_KEY`	Env not loaded	Check `.env.production` and the compose `env_file`
Migration error	A migration failed and rolled back	Restore the backup, fix the migration, redeploy
`Schema drift` warning	Model column missing in DB	Add the migration
Database is locked	A second process is using the file	Ensure one API container; stop any stray process

11.2 Runs stay Pending

1. `GET /api/jobs?kind=pipeline.run`: is the job `queued`, `running` or `failed`?
2. `queued` for a long time: worker not running or concurrency saturated; check logs for handler errors.
3. `failed`: read the job `error`; common causes are an invalid or exhausted Serper key (401, 403, 429) or no LLM key.
4. After a restart, `running` jobs are re-queued automatically; a resumed run repeats work from the start.

11.3 Leads show “Enrichment failed”

The LLM call failed or returned unparsable output. The lead is kept with a reason. Check provider keys and rate limits, then use Retry on the lead (queues `lead.retry_enrichment`).

11.4 A mailbox stopped sending

1. Look for `paused` with a `pause_reason`. A hard-bounce rate above 2% after 20 sends pauses it.
2. Fix the cause (clean the list, check SPF, DKIM and DMARC with the DNS preflight), then Resume.
3. If not paused: confirm the send window, daily cap or warmup ramp is not the limit, and that the contacts are not suppressed.
4. Use “Test connection” to confirm SMTP and IMAP credentials.

11.5 Payments did not credit

Credits are added only when the provider’s signed webhook arrives. Check the provider dashboard for delivery failures, confirm `PUBLIC_API_URL` and the webhook secret, and look at `billing_events`. Replays are safe: processing is idempotent per event.

11.6 Handle an opt-out or erasure request

1. Add the address or domain under Compliance, Suppressions (or the recipient uses the public opt-out link).

2. For erasure, use the Compliance erase tool with the person's email; confirm the audit entry.
3. Remember backups: restore procedures must re-apply erasures made since the backup.

12. Release procedure

1. All checks green: `uv run ruff check .`, `uv run pytest`, `npm run lint`, `npx vitest run`, `npm run build` (CI runs the same).
2. Back up the volume.
3. `docker compose up --build -d`.
4. Check `/api/health`, log in, open a recent request, and confirm the job list is healthy.
5. Roll back by redeploying the previous image and restoring the backup if a migration ran.

13. Pre-hosting checklist

- ☐ Legal review complete; privacy notice and terms published (see document 04)
- ☐ `OUTREACH_POSTAL_ADDRESS` set
- ☐ TLS in front; HSTS and security headers added at the proxy
- ☐ Versioned data-encryption key and a secret manager for `SECRET_KEY`
- ☐ Backups scheduled and a restore tested
- ☐ Retention purge scheduled
- ☐ Monitoring and uptime checks in place
- ☐ End-to-end run with real keys: pipeline, verification providers, SMTP and IMAP, Stripe or Razorpay, HubSpot
- ☐ Two-account check of shared lists and requests in one workspace
- ☐ Dependency and secret scanning enabled in CI